Game Design: Gott einer Zivilisation

Arbeitsstand, wird fortlaufend ergänzt · Projekt: MyWorldProject · Stand: 29.10.2026

1. Kernidee

Der Spieler ist der Gott einer Zivilisation und kann das Geschehen nur **indirekt** beeinflussen. Die eigene und alle anderen Zivilisationen handeln grundsätzlich unabhängig. Betet die eigene Zivilisation den Spieler an, erhält er **Einflusspunkte**, die er in **Gachas** einsetzt, um **Fähigkeiten** zu erhalten. Langfristig kann er damit Zivilisationen und Naturkatastrophen erschaffen oder die Welt verändern.

2. Kernschleife

1. Die Zivilisationen entwickeln sich autonom (Wachstum, Handel, Konflikte, Glaube).
2. Aus Not und Dankbarkeit der Bewohner entstehen Gebete.
3. Anbetung ergibt Einflusspunkte.
4. Einflusspunkte werden in Gachas eingesetzt und liefern Fähigkeiten.
5. Fähigkeiten greifen indirekt in die Welt ein und verändern das Verhalten der Zivilisationen.

3. Zeit und Darstellung

- **Zeit:** Zeitraffer mit Pause und Geschwindigkeitsstufen. Die Simulation läuft nur bei geöffneter App, es gibt keinen Offline-Fortschritt.
- **Darstellung:** 2D-Karte aus **Hexfeldern**.
- **Rolle des Spielers:** Der Spieler beobachtet vor allem, wie sich die Welt bewegt und entwickelt. Eingreifen kann er nur indirekt über Fähigkeiten.

4. Welt

- Die Karte wird bei jedem neuen Spiel **prozedural generiert**.
- Biome (z. B. Ebene, Wald, Gebirge, Wüste, Küste, Wasser) und Ressourcen bestimmen, wo Siedlungen gedeihen.
- Die Karte muss später veränderbar sein (Katastrophen, Weltveränderung durch Fähigkeiten).

5. Zivilisationen

- **Start:** Zu Spielbeginn existiert eine Zivilisation, der Stamm des Spielers. Weitere Zivilisationen entstehen im Laufe des Spiels, bis zu **3–6** gleichzeitig, jeweils mit eigener Persönlichkeit und Tiefe.
- Sie unterscheiden sich durch **Gelände und Kultur**: Das Gelände formt die Kultur, die Kultur formt das Verhalten.
- Sie handeln unabhängig (siedeln, handeln, Krieg, Bündnisse, Wanderung). Sie wachsen und entscheiden selbst, wohin sie ziehen.

6. Bewohner und Persönlichkeiten

- Die Simulation arbeitet auf **Bewohnerebene** mit über **10.000** Agenten.
- Die Masse läuft als vereinfachte Hintergrund-Agenten.
- **Lebenszyklus:** Bewohner altern, bekommen Kinder und sterben. Glaube kann vererbt werden.
- **Bedürfnisse:** umfassend, von den Grundbedürfnissen bis zu Sinnsuche und Spiritualität.
- **Charakter:** Alle Charaktere haben Eigenschaften und Merkmale (Traits).
- **Besondere Persönlichkeiten** (Propheten, Herrscher, Helden) sind einzeln einsehbar und die Hebel der Geschichte. Sie entwickeln sich tiefer weiter und haben eine Geschichte. Sie entstehen meist von selbst, der Spieler kann sie fördern oder anstoßen.

7. Glaube und Anbetung

- Jeder Bewohner hat einen individuellen **Glaubenswert**.
- Gebete entstehen aus **Not und Dankbarkeit**.
- Der Glaubenswert ändert sich durch eigene Erfahrungen (z. B. Wunder, Not, erfüllte oder enttäuschte Hoffnung) und durch das Umfeld (Familie, Nachbarn, Propheten).
- **Einflusspunkte:** Die Menge zählt, jedes Gebet bringt etwa gleich viel.
- **Start:** Der Spieler beginnt mit einem kleinen Stamm von **20 Gläubigen** und einem festen Startvorrat an Einflusspunkten. Später entstehende Zivilisationen starten ohne Gott. Sie können an den Spieler glauben oder einen eigenen Glauben formen.
- Offen: Balance der Einflussmenge, da sie mit der Zahl der Gläubigen wächst.

8. Rivalen-Götter

- Fremde Glaubensrichtungen beginnen als **Ideen**.
- Bei genügend Gläubigen entwickeln sie sich zu **aktiven Göttern**.
- Aktive Rivalen-Götter folgen **denselben Regeln wie der Spieler** (Anbetung, Einflusspunkte, Gacha, Fähigkeiten).
- **Zeitplan:** späteres Ziel, nicht Teil der ersten spielbaren Version.

9. Spielrahmen

- **Sandbox** ohne festes Ziel, die Geschichte ist das Spiel. Eine Partie soll so lange wie möglich spielbar sein.
- **Spielende:** Gibt es keinen Gläubigen mehr, der den Spieler anbetet, endet das Spiel. Diese Regel kann sich später noch ändern.
- Kein Meta-Fortschritt zwischen Durchläufen (Stand der Entscheidungen).
- **Speichern:** Das Spiel speichert regelmäßig, automatisch und effizient, da Partien sehr lange laufen.

10. Gacha und Fähigkeiten (Grundsystem steht)

- **Währung:** ausschließlich Einflusspunkte aus Anbetung, kein Echtgeld.
- **Wirkung:** Jeder Gacha-Gewinn ist eine verbrauchbare Ladung, die beim Einsatz verbraucht wird.

- **Pools:** vier Pools nach Themen: Natur (Wetter, Ernte, Tiere, Gelände), Glück und Schicksal (Zufall, Begegnungen, Wendungen), Krieg (Mut, Waffen, Schutz, Schärfung von Konflikten) und Geist (Glaube, Eingebungen, Weisheit, Propheten). Der Spieler wählt den Pool.
- **Zufall:** starke Zufallskomponente, die Überraschung ist der Reiz. Kein Pity-System.
- **Rhythmus:** wenige, teure Ziehungen. Jede Ziehung ist ein gewichtiger Moment.
- **Seltenheiten:** gewöhnlich, selten, episch, legendär. Empfehlung: Auch die niedrigste Stufe muss nützlich sein, da bei wenigen Ziehungen und starkem Zufall jedes Ergebnis zählt.
- **Ziele:** Je nach Fähigkeit wirkt sie auf Orte (Hexfeld, Region) oder auf Personen (z. B. einen Propheten).
- Offene Punkte: konkrete Fähigkeiten je Pool und Seltenheit, Kosten pro Ziehung, Höhe des Start-Einflussvorrats und Einsatzregeln (Reichweite, Dauer).

11. Technik (vorläufig)

Die Simulation ist bei über 10.000 Agenten machbar, aber auf dem JavaScript-Thread nicht unbegrenzt: Unter Hermes läuft ein Prototyp-Tick etwa 13-mal langsamer als unter Node (V8).

Agenten	Node/V8 (ms/Tick)	Hermes (ms/Tick)	Hermes (Ticks/s max)
10.000	1,05	13,84	72
20.000	1,99	28,66	35
50.000	5,63	70,66	14

Gemessen wurde ein vereinfachter Prototyp (1 Tick = 1 Woche; Bedürfnisse, Gebete, Glaubensausbreitung, Bewegung, jährlich Altern, Tod und Geburt, 300 Persönlichkeiten) auf einer Server-CPU (Xeon 2,8 GHz) mit einer älteren Hermes-Version (0.12). Auf Handys können die Werte je nach Gerät besser oder schlechter ausfallen, es sind Richtwerte. Der echte Spielkern wird eher schwerer (Handel, Konflikte, Diplomatie).

- **Laufzeit:** Der Simulationskern läuft nativ in C++ und wird über JSI angebunden.
- **Vorgehen:** Die Spielregeln werden zuerst in TypeScript entworfen und getestet. Der native C++-Kern wird portiert, sobald die Regeln stabil sind. Der TypeScript-Kern bleibt als Referenz.
- **Update-Strategie:** Die Masse der Bewohner wird gestaffelt aktualisiert (reihum, mit größerem Zeitschritt), Persönlichkeiten in jedem Tick.
- **Kartenrendering:** Skia (react-native-skia).
- Simulationskern von der React-Native-Oberfläche trennen, deterministisch per Seed pro Welt, Daten als flache Arrays.
- Der Simulationsstand lebt außerhalb von React. React Context hält nur UI-nahen Zustand.
- Die Karte bekommt kompakte Snapshots (z. B. Glaube und Dichte je Hexfeld), nicht einzelne Bewohner.
- Speicherstand: regelmäßiges Autosave, kompakt und schnell. Offen: genaues Format (binär statt JSON) und Speicherintervall.

- Umsetzung: React Native mit Expo (Dev Builds/Prebuild), TypeScript strict, React Context + Hooks.
- **Zielplattform:** Android.
- **Spielsprache:** Englisch.

12. Nächste Schritte

1. Bewohner und Persönlichkeiten ausarbeiten.
2. Konkrete Fähigkeiten je Pool und Seltenheit ausarbeiten.
3. Zivilisationen und Welt bei Bedarf verfeinern.
4. Technische Architektur ableiten.